

# Evolutionary Computing

## Lokale Suche

Prof. Dr. Veronika Lesch

INF23

## Copyright

Alle Materialien in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Vorlesungs- und Übungsmaterialien an Dritte ist generell untersagt!

# Gliederung

- 1 Lokale Suche
- 2 Lokale Suche
- 3 Der Bergsteiger
- 4 Tabu Suche
- 5 Simulated Annealing
- 6 Quellen

# Lokale Suche



# Lokale Suche

# Grundsätze lokaler Suche [4]

**In realen Problemen ist die zu optimierende Funktion i.d.R. nicht bekannt!**

- **Blackbox-Funktion:**  $f$  liefert nur einen Wert – kein Gradient, keine Ableitung, keine Struktur
- **Sampling:** Einzige Erkenntnisquelle ist das Auswerten von Lösungskandidaten
- **Unbekannte Landschaft:** Suchraum  $\mathcal{X}$  und Oberfläche von  $f$  sind nicht bekannt
- **Lösungskandidat:** Jeder getestete Punkt  $x \in \mathcal{X}$  heißt *Kandidat* – er repräsentiert eine potenzielle Lösung

**Lokale Suche versucht iterative Verbesserung durch Testen von Lösungskandidaten.**

# Prinzip der Lösungskandidaten [4]

**Repräsentation = Kodierung eines Kandidaten  $x \in \mathcal{X}$ :**

- **Reell / Numerisch:**  
 $x = (x_1, \dots, x_d) \in \mathbb{R}^d$
- **Ganzzahlig:** z. B. Scheduling, 8-Damen
- **Binär:**  $x \in \{0, 1\}^n$ , Rucksack
- **Kategorial:** Farbe, Material
- **Vektor / Permutation:** Reihenfolge von Städten (TSP)
- **Baumstruktur:** Programme, Ausdrücke
- **Graph:** Netzwerktopologien
- **Regelmenge:** Klassifikationsregeln, Strategien

**Die Wahl der Repräsentation bestimmt, welche Operatoren möglich sind.**

# Voraussetzungen für lokale Suche [4]

## Jeder lokale Suchalgorithmus benötigt vier Bausteine:

### 1. Initialisierung

Erzeuge  
einen/mehrere  
Startpunkt  $x_0 \in \mathcal{X}$   
z. B. zufällig,  
heuristisch

### 2. Bewertung

Berechne  $f(x)$  für  
einen Kandidaten  
Ggf. teuer –  
Simulation,  
Experiment

### 3. Kopieren

Erstelle eine Kopie  
 $x' \leftarrow x$   
Wichtig: keine  
Referenz!

### 4. Modifikation

Verändere  $x'$   
 $\text{Tweak}(x')$   
Bewegung durch  
Nachbarschaft  $\mathcal{X}$

# Die Nachbarschaft - Tweak Funktion

**Tweak( $x$ )**: Erzeugt aus einer Lösung  $x$  einen **Nachbarn**  $x'$  durch eine kleine, zufällige Veränderung – ein Schritt im Lösungsraum.

## Beispiele je nach Problemtyp:

- **Kontinuierliche Funktion**: Addiere kleines  $\delta \sim \mathcal{U}(-\epsilon, \epsilon)$  auf eine oder mehrere Koordinaten:  $x' = x + \delta$
- **Rucksack Problem**: Flippe ein zufälliges Bit – nehme einen Gegenstand heraus oder lege einen hinein:  $x'_i = 1 - x_i$
- **8-Damen Problem**: Verschiebe eine zufällig gewählte Dame um ein Feld innerhalb ihrer Spalte

# Der Bergsteiger

# Der Bergsteiger (Hill Climbing) [4]

## Der einfachste lokale Suchalgorithmus

### Grundidee:

- Starte mit einem Kandidaten  $x$
- Erzeuge eine leicht modifizierte Kopie  $x' = \text{Tweak}(x)$
- Übernimm  $x'$  nur, wenn der Wert besser ist:  $f(x') \geq f(x)$
- Wiederhole – bis kein Fortschritt mehr möglich ist

**Analogie:** Ein Wanderer im Nebel, der immer den nächsten Schritt bergauf wählt – ohne die Gesamtlandschaft zu sehen.

# Der Bergsteiger – Pseudocode [4]

---

## Algorithm 1 Hill Climbing

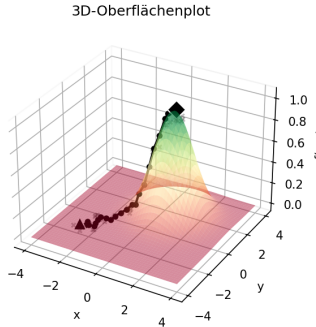
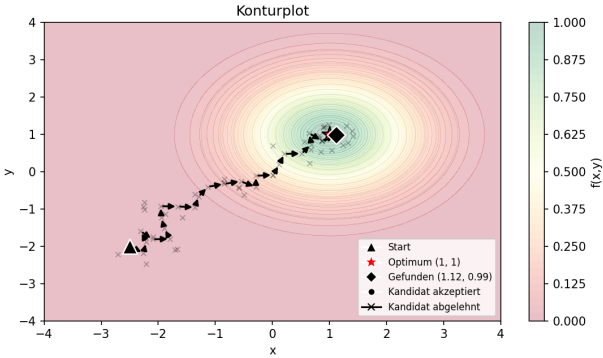
---

```
1: function HILLCLIMBING( $f$ ,  $n_{\max}$ )
2:    $x \leftarrow \text{INITIALIZE}$ 
3:    $i \leftarrow 0$ 
4:   repeat
5:      $x' \leftarrow \text{TWEAK}(x)$ 
6:     if  $f(x') \geq f(x)$  then
7:        $x \leftarrow x'$ 
8:      $i \leftarrow i + 1$ 
9:   until  $x'$  is ideal solution or  $i \geq n_{\max}$ 
10:  return  $x$ 
```

---

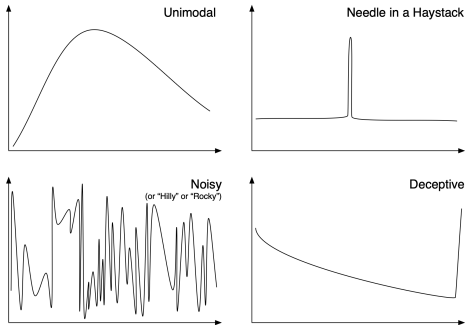
# Der Bergsteiger (Hill Climbing) [4]

Local Hill Climber auf unimodaler Gauß-Funktion



Ziel: Maximierung | Optimum: (1, 1),  $f = 1.0$  | Gefunden: (1.12, 0.99),  $f = 0.9931$  | Akzeptierte Schritte: 24 | Geprüfte Kandidaten: 60

# Gruppenarbeit – Grenzen des Bergsteigers



[4]

- Für welche Fälle funktioniert Hill Climbing? Wo versagt Hill Climbing und warum?
- Entwickeln Sie für die problematischen Fälle Verbesserungsideen
- Coding Aufgabe: Überlegen Sie sich eine 2-dimensionale Funktion, bei der Hill Climbing gut funktioniert, und eine, bei der er schlecht funktioniert. Visualisieren Sie die Fitnesslandschaften in Python.

---

## Algorithm 2 Steepest Ascent Hill Climbing

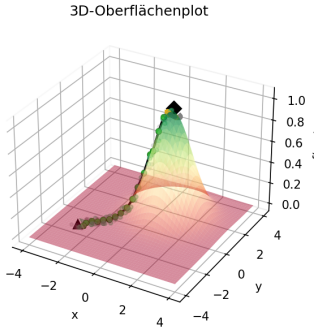
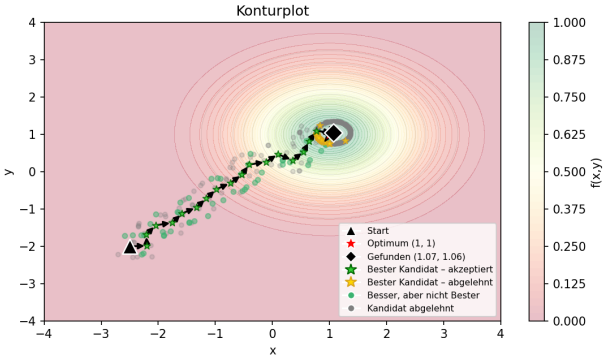
---

```
1: function STEEPESTASCENTHILLCLIMBING( $f, n_{\max}, n$ )
2:    $x \leftarrow \text{INITIALIZE}$ 
3:    $i \leftarrow 0$ 
4:   repeat
5:      $x' \leftarrow \text{TWEAK}(x)$ 
6:     for  $n - 1$  times do
7:        $x'' \leftarrow \text{TWEAK}(x')$ 
8:       if  $f(x'') > f(x')$  then
9:          $x' \leftarrow x''$ 
10:    if  $f(x') > f(x)$  then
11:       $x \leftarrow x'$ 
12:  until  $x$  is ideal or  $i \geq n_{\max}$ 
13:  return  $x$ 
```

---

# Der Bergsteiger (Hill Climbing) [4]

**Steepest-Ascent Hill Climber auf unimodaler Gauß-Funktion (N = 8 Nachbarn / Schritt)**



Ziel: Maximierung | Optimum: (1, 1),  $f = 1.0$  | Gefunden: (1.07, 1.06),  $f = 0.9961$  | Akzeptierte Schritte: 18 | Geprüfte Kandidaten: 480 (8/Schritt)

# Typische Erweiterungen des Bergsteigers [4]

## Random Restart

- Starte Hill Climbing  $k$ -mal von zufälligen Punkten
- Behalte das globale Beste
- Einfach, parallelisierbar
- Reduziert Abhängigkeit vom Startpunkt

## Rückschritte erlauben

- Akzeptiere gelegentlich schlechtere Lösungen
- Kontrolliert durch Temperatur / Wahrscheinlichkeit
- Basis für *Simulated Annealing*

## Tabu-Liste / Besuchte Punkte speichern

- Bereits besuchte Kandidaten werden markiert und ausgeschlossen
- Verhindert Zyklen und Wiederholungen
- Grundprinzip der *Tabu Search*

# Lokale vs. Globale Optima

## Lokales Optimum $x_{\text{lok}}^*$

- $f(x_{\text{lok}}^*) \geq f(x)$  für alle  $x$  in der *Nachbarschaft*
- Hill Climbing terminiert hier – kann nicht entkommen
- Qualität hängt von der Landschaft ab

## Globales Optimum $x^*$

- $f(x^*) \geq f(x)$  für alle  $x \in \mathcal{X}$
- Suchziel – aber meist unbekannt

**Kein deterministischer Algorithmus mit polynomielltem Aufwand garantiert das globale Optimum einer beliebigen Blackbox.**



[1]

- Aufgabe 1: Game-Tuning per Hill Climbing
- Aufgabe 2: Lieferdienst-Routing

# Tabu Suche

## Was wäre, wenn der Algorithmus ein Gedächtnis hätte?

### Kernidee:

- Erlaube auch verschlechternde Schritte – um lokale Optima zu verlassen
- Führe eine **Tabuliste**  $L$ : zuletzt besuchte Lösungen sind verboten
- Verhindere so, dass der Algorithmus im Kreis läuft (Cycling)

**Analogie:** Hänsel und Gretel, die sich ihren Weg durch den Wald mit Brotrümeln markieren. Die entferntesten Brotrümel werden von den Tieren im Wald gefressen.

# Tabu Suche - Von der Idee zum Algorithmus

## Ablauf:

- Erzeuge  $n$  Nachbarn  $x'$  von der aktuellen Lösung  $x$
- Wähle den besten Nachbarn – auch wenn er schlechter als  $x$  ist
- Nur Bedingung:  $x' \notin L$  (nicht auf der Tabuliste)
- Füge  $x'$  zur Liste  $L$  hinzu, ältester Eintrag fliegt raus

## Zwei Steuerparameter:

- $l$  – Länge der Tabuliste: kontrolliert das „Gedächtnis“
- $n$  – Anzahl der Nachbarn: kontrolliert die lokale Exploration

# Tabu Suche – Pseudocode [4]

---

## Algorithm 3 Tabu Search( $f, n_{\max}, l, n$ )

---

```

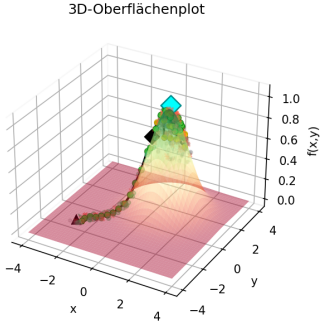
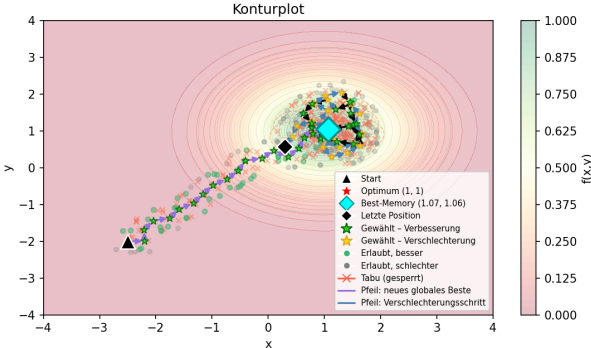
1:  $x, x_{\text{best}}, L \leftarrow \text{INITIALIZE}$ 
2: repeat
3:   if  $|L| > l$  then
4:      $L \leftarrow L[2:]$                                 ▷ Lösche ältestes Tabu Element
5:    $x' \leftarrow \text{TWEAK}(x)$ 
6:   for  $n - 1$  times do                                ▷ Suche besten erlaubten aus  $n - 1$  Nachbarn
7:      $x'' \leftarrow \text{TWEAK}(x)$ 
8:     if  $x'' \notin L$  and  $(f(x'') > f(x'))$  or  $x' \in L$  then
9:        $x' \leftarrow x''$ 
10:    if  $x' \notin L$  then                                ▷ Ersetze ggf. Ausgangspunkt  $x$  und ergänze Tabu-Liste
11:       $x \leftarrow x'$ 
12:       $L \leftarrow L \cup \{x'\}$ 
13:    if  $f(x) > f(x_{\text{best}})$  then                        ▷ Ersetze ggf. bisher besten Kandidaten
14:       $x_{\text{best}} \leftarrow x$ 
15: until  $x_{\text{best}}$  is ideal or  $i \geq n_{\max}$ 
16: return  $x_{\text{best}}$ 

```

---

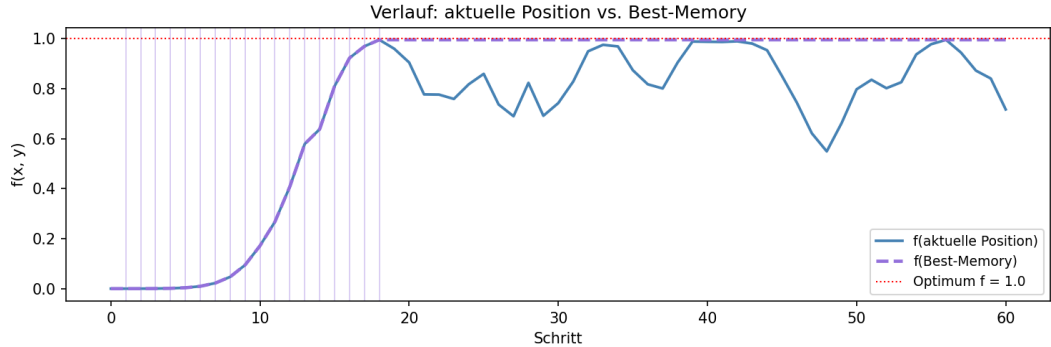
# Tabu Suche

**Tabu Search + Best-Memory auf unimodaler Gauß-Funktion (N=8, Tenure=10, Radius=0.4)**



Ziel: Maximierung | Optimum: (1, 1),  $f = 1.0$  | Bestes gefundenes  $f = 0.9961$  @ (1.07, 1.06) | Neue-Beste-Updates: 18 | Verschlechternde Schritte: 25 | Tabu-Kandidaten: 267

# Tabu Suche



# Tabu Suche – Stärken, Grenzen & Ausblick

## Stärken

- Entkommt lokalen Optima durch erlaubte Verschlechterungen
- Einfach implementierbar – wenige Steuerparameter ( $l$ ,  $n$ )
- Flexibel: funktioniert auf diskreten *und* kontinuierlichen Suchräumen

## Grenzen

- Parameterempfindlich: falsches  $l$  führt zu Cycling oder eingeschränkter Suche
- Tabuliste speichert Lösungen – bei großen Suchräumen teuer oder unpräzise
- Kein probabilistisches Prinzip: jeder Schritt ist deterministisch  
⇒ identische Starts liefern identische Ergebnisse

# Simulated Annealing

# Simulated Annealing – Idee [3]

**Was wäre, wenn der Algorithmus manchmal absichtlich einen schlechteren Schritt wählt?**

## Analogie: Metallabkühlung

- Erhitze Metall stark  
⇒ Atome bewegen sich frei
- Kühle schnell ab  
⇒ Atome frieren in ungünstiger Position ein
- Kühle langsam ab  
⇒ Atome finden energiearme, stabile Gitterstruktur

## Übertragung auf Optimierung

- Hohe Temperatur  $T$   
⇒ Verschlechterungen oft akzeptiert  
– umfangreiches Testen
- Sinkende Temperatur  
⇒ Akzeptanz nimmt ab – mehr Fokus auf aktuellen „Berg“
- $T \rightarrow 0$   
⇒ Verhalten wie reines Hill Climbing

# Simulated Annealing – Akzeptanzkriterium

## Fall 1: Verbesserung

$f(x') > f(x) \Rightarrow$  **immer** akzeptieren

## Fall 2: Verschlechterung

$f(x') \leq f(x) \Rightarrow$  akzeptiere mit

Wahrscheinlichkeit

$$P = e^{\frac{f(x') - f(x)}{T}}$$

## Wie verhält sich $P$ ?

- Hohe Temperatur  $T \Rightarrow P \rightarrow 1$  (fast alles wird akzeptiert)
- Niedrige Temperatur  $T \Rightarrow P \rightarrow 0$  (kaum Verschlechterungen)
- Kleine Verschlechterung  $\Rightarrow P$  groß (wird oft akzeptiert)
- Große Verschlechterung  $\Rightarrow P$  klein (wird selten akzeptiert)

# Simulated Annealing – Steuerparameter

| Parameter | Bedeutung       | Auswirkung                                                                                                                                                                                                                |
|-----------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $T_0$     | Starttemperatur | <p>Hoch: breite Exploration, viele Verschlechterungen akzeptiert</p> <p>Niedrig: kaum Exploration, Gefahr früher Konvergenz</p>                                                                                           |
| $\alpha$  | Abkühlrate      | <p>Klein (<math>\approx 0.8</math>): schnelle Abkühlung, weniger Exploration</p> <p><math>T \leftarrow \alpha \cdot T, \alpha &lt; 1</math>    Groß (<math>\approx 0.99</math>): langsame Abkühlung, bessere Qualität</p> |

**Faustregel:** Ein guter Abkühlplan lässt  $T$  langsam genug sinken, sodass der Algorithmus jeden Bereich des Suchraums ausreichend erkunden kann.

# Simulated Annealing – Pseudocode [4]

---

**Algorithm 4** Simulated Annealing( $f, n_{\max}, T_0, \alpha$ )

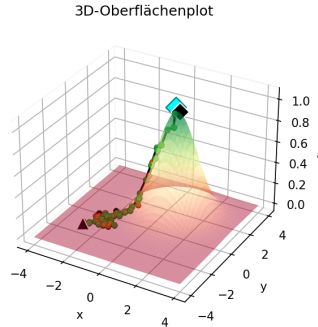
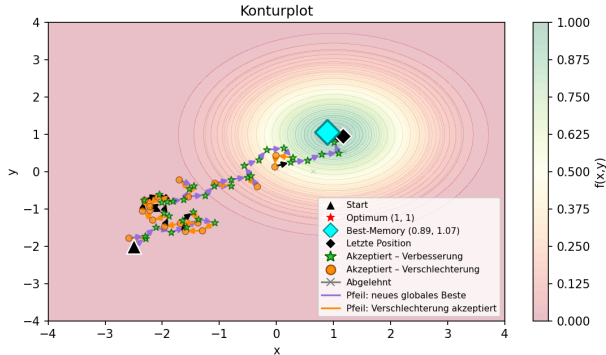
---

```
1:  $T \leftarrow T_0$  ▷ Temperatur, initial hoch
2:  $x \leftarrow \text{INITIALIZE}$ 
3:  $x_{\text{best}} \leftarrow x$ 
4: repeat
5:    $x' \leftarrow \text{TWEAK}(x)$ 
6:   ▷ Verbesserung immer, Verschlechterung probabilistisch akzeptieren
7:   if  $f(x') > f(x)$  or  $\text{random}(0, 1) < e^{(f(x') - f(x)) / T}$  then
8:      $x \leftarrow x'$ 
9:    $T \leftarrow \alpha \cdot T$  ▷ Temperatur abkühlen
10:  if  $f(x) > f(x_{\text{best}})$  then
11:     $x_{\text{best}} \leftarrow x$ 
12: until  $x_{\text{best}}$  is ideal,  $i \geq n_{\max}$  or  $T \leq 0$ 
13: return  $x_{\text{best}}$ 
```

---

# Simulated Annealing - Beispiel

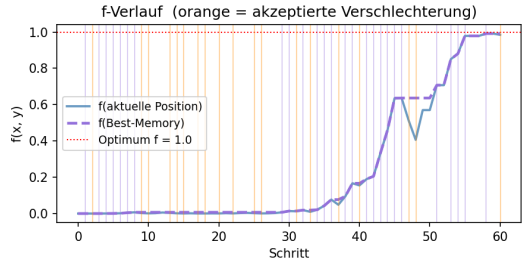
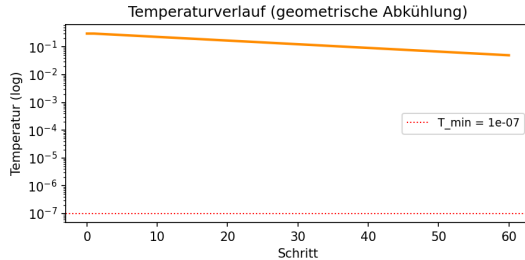
**Simulated Annealing auf unimodaler Gauß-Funktion ( $T_0=0.3$ ,  $\alpha=0.97$ ,  $T_{\min}=1e-07$ )**



Ziel: Maximierung | Optimum: (1, 1),  $f = 1.0$  | Best-Memory:  $f = 0.9917$  @ (0.89, 1.07) | Verschl. akzeptiert: 19 | Verschl. abgelehnt: 5 | Neue-Beste-Updates: 26

# Simulated Annealing - Verlauf

## Simulated Annealing - Temperatur & Akzeptanzverhalten



# Simulated Annealing – Stärken, Grenzen & Ausblick

## Stärken

- Probabilistisch: entkommt lokalen Optima ohne explizites Gedächtnis
- Theoretisch konvergiert SA gegen das globale Optimum – bei geeignetem Abkühlplan
- Wenige Parameter

## Grenzen

- Abkühlplan ist entscheidend – schlechte Wahl verschenkt Konvergenz
- Kein Gedächtnis: bereits gute Lösungen können wieder verlassen werden
- Langsam bei sehr rauem Suchraum: viele Iterationen bei niedriger Temperatur nötig



[1]

- Aufgabe 3: Liefersdienst-Routing mit Tabu Suche
- Aufgabe 4: Liefersdienst-Routing mit Simulated Annealing

# Showcase - Lokale Suche



[1]

- Entscheiden Sie sich für einen lokalen Suchalgorithmus Ihrer Wahl
- Implementieren Sie den Algorithmus in Python mit dieser Fitnessfunktion

$$f(x, y) = e^{-\frac{(x+3)^2 + (y-2)^2}{2}}$$

- Evaluieren Sie die zu prüfenden Punkte mit dem Webtool
- Danach vergleichen wir Ihre Algorithmen über das Webtool und weitere unbekannte Funktionen

## Quellen

- [1] Generiert von Microsoft Copilot.  
[Online; accessed 2026-03-12].
  
- [2] Fred Glover.  
Future paths for integer programming and links to artificial intelligence.  
*Computers & Operations Research*, 13(5):533–549, 1986.  
Applications of Integer Programming.
  
- [3] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi.  
Optimization by simulated annealing.  
*Science*, 220(4598):671–680, 1983.
  
- [4] Sean Luke.  
*Essentials of Metaheuristics*.  
Lulu, second edition, 2013.  
Available at <https://people.cs.gmu.edu/~sean/book/metaheuristics/>.